

PPO Implementation in Flax

Key Foundational Concepts in Policy Gradient Methods

What are Policy Gradient Methods?

Policy gradient methods are a family of reinforcement learning algorithms that directly optimize the policy function $\pi(a|s)$ - the probability distribution over actions given states. Unlike value-based methods (like Q-learning) that learn action values and derive policies indirectly, policy gradient methods parameterize the policy directly and use gradient ascent to maximize expected cumulative reward.

From Policy Gradient Theorem to State-of-the-Art GAE Implementation

This section presents a concise mathematical journey from the foundational policy gradient theorem to the sophisticated Generalized Advantage Estimation (GAE) used in modern PPO implementations, with particular focus on the bias-variance tradeoff that drives these developments.

Step 1: Policy Gradient Foundation

Core Objective: Maximize expected return: $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$

Key Assumptions: - **MDP Framework:** States S , actions A , transitions $P(s'|s,a)$, rewards $R(s,a)$, discount γ - **Differentiability:** Policy $\pi(a|s)$ differentiable w.r.t. θ - **Stationarity:** Time-invariant dynamics and policy (momentarily during gradient computation)

Policy Gradient Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) R_t \right]$$

Why Do We Use Log Probability?

The use of log probability in the policy gradient theorem is fundamental and arises from several mathematical and practical considerations:

1. Likelihood Ratio Trick Application: The log probability emerges naturally from the likelihood ratio trick. When we want to compute the gradient of an expectation, we use:

$$\nabla_\theta \mathbb{E}_{x \sim p_\theta}[f(x)] = \mathbb{E}_{x \sim p_\theta}[f(x) \nabla_\theta \log p_\theta(x)]$$

This transformation is possible because:

$$\nabla_\theta p_\theta(x) = p_\theta(x) \nabla_\theta \log p_\theta(x)$$

2. Numerical Stability: Working with log probabilities provides better numerical stability:
 - Probabilities can be very small (close to 0), leading to numerical underflow - Log probabilities convert multiplication to addition: $\log(ab) = \log(a) + \log(b)$ - Gradients of log probabilities are better behaved numerically

3. Natural Gradient Interpretation: The gradient $\nabla_{\theta} \log \pi_{\theta}(a|s)$ represents the **score function** or **natural gradient direction** for increasing the probability of action a in state s . This provides: - Direct control over probability changes - Proper scaling relative to current probability values - Natural weighting that prevents extreme updates

4. Mathematical Elegance: Log probabilities simplify many calculations: - For softmax policies: $\nabla_{\theta} \log \pi_{\theta}(a|s) = \mathbf{e}_a - \pi_{\theta}(\cdot|s)$ (one-hot minus probability vector) - For Gaussian policies: $\nabla_{\theta} \log \pi_{\theta}(a|s)$ has closed-form expressions - Chain rule applications become more tractable

5. Policy Update Interpretation: The term $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) R_t$ can be interpreted as: - **Direction:** $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$ points toward increasing probability of action a_t - **Magnitude:** R_t determines how much to increase (positive) or decrease (negative) this probability - **Weighting:** Actions leading to higher returns get stronger probability increases

Why R_t Takes This Form: The return R_t represents the cumulative discounted reward from time step t onward:

$$R_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$$

This formulation arises from the **causality principle**: actions at time t can only influence rewards received at time t and later, not earlier rewards. Therefore, each log probability $\log \pi_{\theta}(a_t|s_t)$ is only weighted by future rewards R_t , ensuring that we don't attribute credit for past rewards to current actions.

General Likelihood Ratio Trick: For any function $f(x)$ and probability distribution $p_{\theta}(x)$:

$$\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}}[f(x)] = \mathbb{E}_{x \sim p_{\theta}}[f(x) \nabla_{\theta} \log p_{\theta}(x)]$$

This identity allows us to move the gradient inside the expectation by exploiting:

$$\nabla_{\theta} p_{\theta}(x) = p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x)$$

Mathematical Foundation: Uses likelihood ratio trick and causality principle to derive this unbiased gradient estimator.

Step 2: The High Variance Problem and Baseline Subtraction

Problem: The basic policy gradient suffers from extremely high variance, making learning unstable.

Understanding Variance and Bias in This Context:

Variance refers to how much the gradient estimates fluctuate around their expected value across different trajectory samples. High variance means: - Gradient estimates are inconsistent between different rollouts - Learning becomes unstable with erratic parameter updates - Convergence is slow and may require many more samples

Bias refers to the systematic error between the expected value of our estimator and the true gradient. An unbiased estimator means: - $\mathbb{E}[\hat{\nabla}_\theta J(\theta)] = \nabla_\theta J(\theta)$ (expectation equals true gradient) - On average, our gradient estimates point in the correct direction - No systematic deviation from the optimal update direction

The challenge is that reducing variance often introduces bias, and vice versa.

Solution - Baseline Subtraction Theorem: Any state-dependent baseline $b(s)$ can be subtracted without bias:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) (R_t - b(s_t)) \right]$$

Key Insight: $\mathbb{E}_{a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) b(s)] = 0$ because $\sum_a \nabla_\theta \pi_\theta(a | s) = \nabla_\theta 1 = 0$

Optimal Baseline: The variance-minimizing baseline is approximately the state value function:

$$b^*(s) \approx V^\pi(s) = \mathbb{E}_{a \sim \pi_\theta} [Q^\pi(s, a)]$$

Step 3: Emergence of Advantage Functions

Mathematical Derivation: Using $V^\pi(s)$ as baseline leads naturally to advantage functions:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

Advantage-Based Policy Gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) A^\pi(s_t, a_t) \right]$$

Understanding Unbiasedness of Advantage Estimation:

Unbiasedness in advantage estimation means that the expected value of our advantage estimates equals the true advantage function:

$$\mathbb{E}[\hat{A}^\pi(s, a)] = A^\pi(s, a)$$

This is crucial because: 1. **No Systematic Error:** Our advantage estimates don't consistently over- or under-estimate the true advantage 2. **Correct Policy Updates:** Unbiased

advantages ensure that policy gradient updates point toward genuine improvements 3. **Convergence Guarantees:** Theoretical convergence results rely on unbiased gradient estimates

Critical Property: $\mathbb{E}_{a \sim \pi_\theta}[A^\pi(s, a)] = 0$ (zero-mean), ensuring unbiasedness while providing directional information about action quality.

Mathematical Proof of Zero-Mean Property:

$$\begin{aligned}\mathbb{E}_{a \sim \pi_\theta}[A^\pi(s, a)] &= \mathbb{E}_{a \sim \pi_\theta}[Q^\pi(s, a) - V^\pi(s)] \\ &= \mathbb{E}_{a \sim \pi_\theta}[Q^\pi(s, a)] - V^\pi(s) = V^\pi(s) - V^\pi(s) = 0\end{aligned}$$

This zero-mean property ensures that advantages provide relative action quality information without introducing bias into the policy gradient.

Step 4: Temporal Difference (TD) Estimation

Problem: Direct computation of $Q^\pi(s, a)$ and $V^\pi(s)$ is intractable.

TD Solution: Use Bellman equation to estimate advantages:

$$\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Understanding the Bias-Variance Tradeoff in This Context:

The **bias-variance tradeoff** is fundamental to advantage estimation and represents the tension between two types of estimation error:

Bias in TD Estimation: - **Source:** Using bootstrapped estimates $V(s_{t+1})$ instead of true future returns - **Effect:** Systematic underestimation or overestimation of advantages - **Mathematical:** $\text{Bias} = \mathbb{E}[\hat{A}_t] - A^\pi(s_t, a_t)$ - **When it occurs:** When $V(s) \neq \hat{V}(s)$ due to function approximation errors

Variance in TD Estimation: - **Source:** Stochastic rewards r_t and random transitions to s_{t+1} - **Effect:** High fluctuation in advantage estimates across different samples - **Mathematical:** $\text{Var} = \mathbb{E}[(\hat{A}_t - \mathbb{E}[\hat{A}_t])^2]$ - **When it's high:** With noisy rewards and stochastic environments

The Fundamental Tradeoff: - **Monte Carlo Direction (Low Bias, High Variance):** Using more actual rewards reduces bias but increases variance due to stochastic returns - **TD Direction (High Bias, Low Variance):** Using more bootstrapped estimates reduces variance but increases bias due to value function approximation errors - **Optimal Balance:** Choose the right mix to minimize total estimation error

Key Assumptions: - **Bellman Validity:** $V^\pi(s_t) = \mathbb{E}_\pi[r_t + \gamma V^\pi(s_{t+1}) | s_t]$ - **Markov Property:** Future depends only on current state - **Value Function Accuracy:** $V(s) \approx \hat{V}(s)$

Advantage Approximation: When $V(s) \approx \hat{V}(s)$, then $\delta_t^V \approx A^\pi(s_t, a_t)$

Step 5: The Bias-Variance Tradeoff in Advantage Estimation

Core Dilemma: - **TD Error** ($=0$): $\hat{A}_t = \delta_t \rightarrow$ High bias, Low variance - **Monte Carlo** ($=1$): $\hat{A}_t = \sum_{k=0}^{\infty} \gamma^k \delta_{t+k} \rightarrow$ Low bias, High variance

Bias-Variance Analysis:

High Variance Sources: 1. **Stochastic Rewards:** Random reward realizations create noise 2. **Environment Stochasticity:** Transition randomness amplifies uncertainty 3. **Function Approximation:** Neural network approximation errors 4. **Finite Sampling:** Limited trajectory samples

Bias Sources: 1. **Value Function Approximation:** $V_-(s) \neq V^+(s)$ 2. **Bootstrapping:** Using estimated values instead of true returns 3. **Truncation:** Finite horizon approximation of infinite sums

Step 6: Generalized Advantage Estimation (GAE) - The Optimal Solution

Mathematical Foundation: GAE provides a principled interpolation between bias and variance:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V$$

Recursive Implementation:

$$\hat{A}_t^{GAE} = \delta_t + \gamma \lambda \hat{A}_{t+1}^{GAE}$$

Bias-Variance Control Parameter : - $= 0$: Pure TD $\rightarrow \hat{A}_t = \delta_t$ (high bias, low variance) - $= 1$: Pure MC $\rightarrow \hat{A}_t = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l}$ (low bias, high variance) - **(0,1):** Optimal tradeoff $\rightarrow$ Exponentially weighted combination

Key Assumptions: 1. **Exponential Decay:** Future information becomes less reliable exponentially 2. **Geometric Convergence:** $|\lambda| < 1$ ensures series convergence 3. **Finite Horizon:** Practical truncation at episode boundaries

Step 7: State-of-the-Art Implementation in PPO

Practical GAE Algorithm:

```
def gae_advantages(rewards, terminal_masks, values, discount, gae_param):
    advantages = []
    gae = 0.0
    for t in reversed(range(len(rewards))):
        # TD error: _t = r_t + V(s_{t+1}) - V(s_t)
        delta = rewards[t] + discount * values[t + 1] * terminal_masks[t] - values[t]
        # GAE recursion: A_t = _t + A_{t+1}
        gae = delta + discount * gae_param * terminal_masks[t] * gae
    advantages.append(gae)
```

```

advantages.append(gae)
return jnp.array(advantages[::-1])

```

Why the Reversed For Loop is Essential:

The reversed for loop (`for t in reversed(range(len(rewards)))`) is not just an implementation detail—it's mathematically necessary for the GAE algorithm to work correctly. Here's why:

1. Mathematical Dependency Structure: The GAE recursive formula shows that each advantage estimate depends on the future:

$$\hat{A}_t^{GAE} = \delta_t + \gamma \lambda \hat{A}_{t+1}^{GAE}$$

This creates a **backward dependency chain**: - $\hat{A}_t$ depends on $\hat{A}_{t+1}$ - $\hat{A}_{t+1}$ depends on $\hat{A}_{t+2}$ - And so on...

2. Boundary Condition: The recursion starts from the **terminal condition**:

$$\hat{A}_T^{GAE} = \delta_T = r_T + \gamma \cdot 0 - V(s_T) = r_T - V(s_T)$$

At the final time step T , there's no future advantage ($\hat{A}_{T+1} = 0$), so we can compute $\hat{A}_T$ directly.

3. Forward Computation is Impossible: If we tried to compute advantages in forward order ($t = 0, 1, 2, \dots$), we would encounter:

$$\hat{A}_0^{GAE} = \delta_0 + \gamma \lambda \hat{A}_1^{GAE}$$

But $\hat{A}_1^{GAE}$ hasn't been computed yet! This creates a circular dependency that cannot be resolved.

4. Backward Computation Resolves Dependencies: Working backwards ($t = T, T-1, T-2, \dots, 0$): - **Step 1:** Compute $\hat{A}_T = \delta_T$ (no dependencies) - **Step 2:** Compute $\hat{A}_{T-1} = \delta_{T-1} + \gamma \lambda \hat{A}_T$ (using known $\hat{A}_T$) - **Step 3:** Compute $\hat{A}_{T-2} = \delta_{T-2} + \gamma \lambda \hat{A}_{T-1}$ (using known $\hat{A}_{T-1}$) - **Continue** until $\hat{A}_0$

5. Computational Efficiency: The backward approach enables **O(T) linear time complexity**: - Each time step is visited exactly once - No need to store intermediate computations - Memory efficient with constant space per time step

6. Terminal State Handling: The `terminal_masks[t]` ensures proper handling of episode boundaries:

```
gae = delta + discount * gae_param * terminal_masks[t] * gae
```

When `terminal_masks[t] = 0` (terminal state), the future contribution is zeroed out, preventing advantages from bleeding across episode boundaries.

7. Array Reversal: The final `advantages[::-1]` reverses the list back to chronological order ($t = 0, 1, 2, \dots, T$) since we appended advantages in reverse temporal order during the backward loop.

Mathematical Intuition: The backward computation reflects the **temporal credit assignment** nature of GAE: - Future rewards and advantages influence current advantage estimates - The `discount` parameter controls how far into the future we look - Each time step “inherits” discounted advantage information from its successor

This backward dependency is fundamental to GAE’s ability to balance bias and variance by incorporating multi-step returns while maintaining computational tractability.

Implementation Assumptions: - **Neural Network Approximation:** $V_-(s) \approx V_+(s)$ using critic network - **Backward Processing:** Efficient $O(T)$ computation via recursion - **Terminal State Handling:** $V(s_{\text{terminal}}) = 0$ - **Batch Processing:** Parallel environment synchronization

Hyperparameter Sensitivity: - **[0.95, 0.99]:** Discount factor (future reward importance)
- **[0.9, 0.99]:** GAE parameter (bias-variance tradeoff)

Step 8: Integration with PPO Loss Function

Final Policy Gradient with GAE:

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t^{GAE} \right]$$

PPO Clipped Objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta) \hat{A}_t^{GAE}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{GAE})]$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$ is the importance sampling ratio.

1. Why Do We Need the Importance Sampling Ratio?

The importance sampling ratio is fundamental to PPO’s ability to reuse experience data for multiple policy updates, addressing a critical challenge in policy gradient methods.

Off-Policy Correction Problem: In the standard policy gradient theorem, we assume that the experience data comes from the current policy π_{θ} :

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]$$

However, in PPO, we collect experience using policy $\pi_{\theta_{old}}$, but then update the policy parameters multiple times. After the first update, the current policy π_{θ} differs from the policy that generated the data $\pi_{\theta_{old}}$.

Mathematical Foundation of Importance Sampling: To correct for this mismatch, we use importance sampling. The key insight is that we can reweight samples from one distribution to estimate expectations under another distribution:

$$\mathbb{E}_{x \sim p}[f(x)] = \mathbb{E}_{x \sim q} \left[\frac{p(x)}{q(x)} f(x) \right]$$

Applied to policy gradients:

$$\mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t] = \mathbb{E}_{\tau \sim \pi_{\theta_{old}}} \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]$$

Surrogate Objective Derivation: The importance sampling ratio allows us to derive the surrogate objective. Starting from the policy gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{old}}} [r_t(\theta) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t]$$

This is equivalent to taking the gradient of:

$$L^{surrogate}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{old}}} [r_t(\theta) \hat{A}_t]$$

Sample Efficiency Benefits: Without importance sampling, we would need to: 1. Collect experience with current policy 2. Perform one gradient update 3. Discard all experience data 4. Repeat from step 1

With importance sampling, we can: 1. Collect experience with $\pi_{\theta_{old}}$ 2. Perform multiple gradient updates using the same data 3. Achieve much better sample efficiency

2. Why Do We Have Both π_{θ} and $\pi_{\theta_{old}}$ at Time t , Given a_t and s_t ?

This question addresses a fundamental aspect of PPO's training procedure and the temporal relationship between policy evaluation and policy improvement.

Experience Collection vs. Policy Update Timeline:

The presence of both $\pi(a_t|s_t)$ and $\pi_{old}(a_t|s_t)$ reflects PPO's two-phase training process:

Phase 1 - Experience Collection: - Use policy π_{old} to interact with environment - For each state s_t , sample action: $a_t \sim \pi_{\theta_{old}}(\cdot|s_t)$ - Store experience tuple: $(s_t, a_t, r_t, \hat{A}_t, \log \pi_{\theta_{old}}(a_t|s_t))$

Phase 2 - Policy Update: - Keep experience data fixed - Update policy parameters: $\pi_{old} \rightarrow \pi$ - For loss computation, evaluate both: - $\pi_{\theta_{old}}(a_t|s_t)$: Probability under the data-collecting policy - $\pi_{\theta}(a_t|s_t)$: Probability under the current (updated) policy

Mathematical Interpretation:

At any given time step t , we have: - **Fixed action a_t** : This was sampled from π_{old} during experience collection - **Fixed state s_t** : This is the environment state when action a_t was taken - $\pi_{old}(a_t|s_t)$: Probability that the old policy would have taken action a_t in state s_t - $\pi(a_t|s_t)$: Probability that the current policy would take action a_t in state s_t

Why Both Are Needed:

1. $\pi_{old}(a_t|s_t)$ (denominator): Represents the “baseline” probability - how likely was this action under the policy that generated the data
2. $\pi(a_t|s_t)$ (numerator): Represents the “current” probability - how likely is this action under the policy we’re optimizing
3. **Ratio $r_t()$** : Measures how much the policy has changed:
 - $r_t() > 1$: Current policy assigns higher probability to a_t than old policy
 - $r_t() < 1$: Current policy assigns lower probability to a_t than old policy
 - $r_t() = 1$: No change in probability for this action

Practical Implementation:

```
# From ppo_lib.py - loss_fn function
# During experience collection (Phase 1):
old_log_probs = log_probs_from_old_policy # pi_old(a_t|s_t)

# During policy update (Phase 2):
log_probs, values = agent.policy_action(apply_fn, params, states) # pi(a_t|s_t)
log_probs_act_taken = jax.vmap(lambda lp, a: lp[a])(log_probs, actions)

# Importance sampling ratio:
ratios = jnp.exp(log_probs_act_taken - old_log_probs) # pi(a_t|s_t) / pi_old(a_t|s_t)
```

3. What's the Relationship Between $\hat{L}^{CLIP}$ and the Final Policy Gradient with GAE?

The relationship between the clipped objective $\hat{L}^{CLIP}$ and the policy gradient with GAE represents the culmination of PPO's theoretical framework, combining variance reduction, sample efficiency, and training stability.

From Policy Gradient to Surrogate Objective:

Starting with the GAE-enhanced policy gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t^{GAE} \right]$$

Using importance sampling to enable off-policy updates:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{old}}} \left[\sum_{t=0}^{T-1} r_t(\theta) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t^{GAE} \right]$$

This gradient corresponds to the surrogate objective:

$$L^{surrogate}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{old}}} \left[\sum_{t=0}^{T-1} r_t(\theta) \hat{A}_t^{GAE} \right]$$

The Clipping Mechanism:

The clipped objective modifies the surrogate objective to prevent destructively large policy updates:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta) \hat{A}_t^{GAE}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{GAE})]$$

Mathematical Analysis of Clipping:

The clipping function operates as follows:

$$\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) = \begin{cases} 1 - \epsilon & \text{if } r_t(\theta) < 1 - \epsilon \\ r_t(\theta) & \text{if } 1 - \epsilon \leq r_t(\theta) \leq 1 + \epsilon \\ 1 + \epsilon & \text{if } r_t(\theta) > 1 + \epsilon \end{cases}$$

Case Analysis:

1. When $\hat{A}_t^{GAE} > 0$ (good action):

- If $r_t(\theta) > 1 + \epsilon$: Clip to $(1 + \epsilon) \hat{A}_t^{GAE}$
- Prevents excessive probability increases for good actions

2. **When $\hat{A}_t^{GAE} < 0$ (bad action):**

- If $r_t(\theta) < 1 - \epsilon$: Clip to $(1 - \epsilon)\hat{A}_t^{GAE}$
- Prevents excessive probability decreases for bad actions

3. **When $1 - \epsilon \leq r_t(\theta) \leq 1 + \epsilon$:**

- No clipping: Use original surrogate objective
- Policy change is within acceptable bounds

Relationship to GAE:

The GAE advantages $\hat{A}_t^{GAE}$ serve multiple roles in the clipped objective:

1. **Direction:** Sign of $\hat{A}_t^{GAE}$ determines whether to increase or decrease action probability
2. **Magnitude:** Absolute value of $\hat{A}_t^{GAE}$ determines the strength of the update
3. **Variance Control:** GAE's bias-variance tradeoff (controlled by γ) affects gradient stability

Final Integration:

The complete PPO loss combines the clipped policy objective with value function learning and entropy regularization:

$$L_{PPO}(\theta) = L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 S[\pi_\theta]$$

where: - $L^{CLIP}(\theta)$: Clipped policy objective using GAE advantages - $L^{VF}(\theta) = \mathbb{E}_t[(\hat{A}_t^{GAE} + V_{\theta_{old}}(s_t) - V_\theta(s_t))^2]$: Value function loss - $S[\pi_\theta] = \mathbb{E}_t[H(\pi_\theta(\cdot|s_t))]$: Entropy bonus

Theoretical Properties:

1. **Conservative Updates:** Clipping ensures policy changes remain bounded
2. **Monotonic Improvement:** Under certain conditions, PPO guarantees policy improvement
3. **Sample Efficiency:** Multiple epochs on the same data with importance sampling correction
4. **Stability:** Clipping prevents policy collapse from large updates

Gradient Relationship: The gradient of L^{CLIP} with respect to θ provides a modified version of the policy gradient that: - Maintains the direction indicated by GAE advantages - Limits the magnitude of updates through clipping - Preserves the unbiasedness properties of the original policy gradient (within the trust region)

Practical Example - Atari Breakout: Consider a 3-step sequence with $\gamma=0.99$, $\beta=0.95$: - **t=0:** $r=0$, $V(s)=15.2$, $V(s)=18.5 \rightarrow \hat{A}=3.115$, $A=4.931$ - **t=1:** $r=1$, $V(s)=18.5$, $V(s)=20.1 \rightarrow \hat{A}=2.399$, $A=1.931$ - **t=2:** $r=0$, $V(s)=20.1$, $V(s)=19.8 \rightarrow \hat{A}=-0.498$, $A=-0.498$

Key Benefits Achieved: 1. **Variance Reduction:** GAE smooths noisy advantage estimates 2. **Bias Control:** parameter allows fine-tuning of bias-variance tradeoff 3. **Sample Efficiency:** Better gradient estimates accelerate learning 4. **Stability:** Reduced gradient noise improves training stability 5. **Credit Assignment:** Multi-horizon consideration improves temporal credit assignment

Theoretical Guarantees: - **Unbiasedness:** When $V(s) = \hat{V}(s)$, GAE provides unbiased advantage estimates - **Consistency:** As $\gamma \rightarrow 1$, GAE approaches Monte Carlo estimates - **Convergence:** Maintains policy gradient theorem guarantees while improving practical performance

This progression from basic policy gradients to sophisticated GAE represents a principled evolution driven by the fundamental bias-variance tradeoff, culminating in the state-of-the-art advantage estimation used in modern PPO implementations.

Evolution of Policy Gradient Methods

1. REINFORCE (1992): The simplest policy gradient algorithm uses the basic policy gradient theorem:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) \cdot R_t^{(i)}$$

Problems with REINFORCE: - High variance in gradient estimates - Sample inefficiency - Unstable training

2. Actor-Critic Methods: Introduced a value function $\hat{V}(s)$ (the “critic”) to reduce variance:

$$\nabla_{\theta} J(\theta) \approx \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot A^{\pi}(s_t, a_t)]$$

where the advantage function $A^{\pi}(s, a) = Q^{\pi}(s, a) - \hat{V}(s)$ measures how much better action a is compared to the average action in state s .

3. Trust Region Methods (TRPO): Addressed the problem of destructively large policy updates by constraining the KL divergence:

$$\max_{\theta} \mathbb{E}[L(\theta)] \text{ subject to } \mathbb{E}[\text{KL}[\pi_{\theta_{old}}, \pi_{\theta}]] \leq \delta$$

where: - $L(\cdot)$ is the surrogate objective function - $\text{KL}[__\text{old}, __]$ is the KL divergence between old and new policies - δ is a trust region constraint

Why PPO Has Been Popular

1. Simplicity and Implementation Ease: PPO replaces TRPO's complex constrained optimization with a simple clipped objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

where: - $r_t() = \frac{a_t|s_t}{a_{old}|s_t}$ is the probability ratio - ϵ is the clipping parameter (typically 0.1-0.3) - $\text{clip}(x, a, b)$ constrains x to the range $[a, b]$

2. Sample Efficiency: PPO reuses collected experience for multiple gradient updates (typically 3-10 epochs), unlike REINFORCE which uses each sample only once. This dramatically improves sample efficiency.

3. Stability: The clipping mechanism prevents destructively large policy updates that could collapse training: - When $r_t() > 1 + \epsilon$ (new policy assigns higher probability), the objective is clipped - When $r_t() < 1 - \epsilon$ (new policy assigns lower probability), the objective is clipped - This ensures the new policy doesn't deviate too far from the old policy

4. Robustness to Hyperparameters: PPO is less sensitive to hyperparameter choices compared to other policy gradient methods, making it more practical for diverse applications.

5. Versatility: PPO works well across different domains: - Continuous and discrete action spaces - Various neural network architectures - Different environment types (games, robotics, etc.)

6. Theoretical Guarantees: Recent work has provided convergence guarantees for PPO under certain conditions, giving theoretical backing to its empirical success.

Key Advantages Over Other Methods

Compared to Value-Based Methods (DQN, etc.): - Naturally handles continuous action spaces - Can learn stochastic policies - More stable in partially observable environments

Compared to REINFORCE: - Much lower variance through multiple epochs and clipping - Better sample efficiency - More stable training

Compared to TRPO: - Simpler implementation (no constrained optimization) - Faster computation (no KL divergence calculations) - Similar performance with less complexity

Compared to A3C/A2C: - Better sample efficiency through experience reuse - More stable training through clipping - Easier to implement and debug

Mathematical Notation Guide

Throughout this document, we use standard reinforcement learning notation:

- $\pi(\mathbf{a}|\mathbf{s})$: Policy function - probability of action $\mathbf{a}$ given state $\mathbf{s}$ with parameters
- $V^\pi(\mathbf{s})$: State value function - expected return starting from state $\mathbf{s}$ following policy
- $Q^\pi(\mathbf{s},\mathbf{a})$: Action value function - expected return starting from state $\mathbf{s}$, taking action $\mathbf{a}$, then following policy
- $A^\pi(\mathbf{s},\mathbf{a})$: Advantage function - $A^\pi(\mathbf{s},\mathbf{a}) = Q^\pi(\mathbf{s},\mathbf{a}) - V^\pi(\mathbf{s})$
- $\mathbf{R}_t$: Return from time t - sum of discounted future rewards
- γ : Discount factor - determines importance of future rewards ($0 \leq \gamma < 1$)
- c : GAE parameter - controls bias-variance tradeoff in advantage estimation
- κ : Clipping parameter - controls how much the policy can change in one update
- ∇_{θ} : Gradient with respect to parameters
- $\hat{\cdot}$: Expected value operator
- τ : Trajectory - sequence of (state, action, reward) tuples
- θ : Policy parameters (neural network weights)